Abstract

This project addresses the challenge of manually generating TikZ code for flowcharts by developing an application that automatically creates TikZ code from a JSON input. Users provide a JSON file based on a defined specification, and the application generates the corresponding TikZ code with a focus on readability. Additionally, the application include a preview feature for the generated code.

By leveraging the Sugiyama framework for hierarchical graph layout, the application simplifies flowchart creation. The project targets academics and students and aims to streamline the process of incorporating graphs into academic papers. This helps in reducing the time spent on learning TikZ, and enables users to concentrate on the content of their work.

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary. I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.

Richmond Bobie

April 12, 2024

Acknowledgements

I would like to thank my supervisor, Dr. Jeroen Keppens, for his support and guidance throughout this project's development and for providing insightful suggestions in response to break down my ideas.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Scope . . . 3
- 1.3 Why Flowchart? . . . 4
- 1.4 Objectives . . . 5

2 Literature **6**
- 2.1 JSON (JavaScript Object Notation) . . . 6
- 2.2 LaTeX & TikZ . . . 6
- 2.3 Visualisation (Graph Drawing) . . . 7

3 Requirements & Specifications **11**
- 3.1 Requirements . . . 11
- 3.2 Specification . . . 13

4 Design **14**
- 4.1 Development Environment . . . 14
- 4.2 JSON Specification . . . 14
- 4.3 Class Diagram . . . 15
- 4.4 Graphical User Interface (GUI) . . . 17
- 4.5 Libraries And Other Applications in Use . . . 18

5 Implementation **21**
- 5.1 MVP 1 - Basic Implementation . . . 21
- 5.2 MVP 2 - Refactoring and Code Improvement . . . 23
- 5.3 MVP 3 - Solution Based on The Sugiyama framework . . . 25
- 5.4 MVP 4 - Graphical User Interface (GUI) Integration . . . 36

6 Legal, Social, Ethical and Professional Issues **38**

7 Evaluation & Testing **39**
- 7.1 Evaluation of algorithms . . . 39
- 7.2 Testing . . . 44
- 7.3 Limitations . . . 46
- 7.4 Overall Project Evaluation . . . 48

8 Conclusion and Future Work **49**
8.1 Conclusion 49
8.2 Future Work 50
Bibliography 52

Chapter 1

Introduction

Over the past two decades, LaTeX has established itself as the standard tool for academic writing, particularly in STEM fields. The broad acceptance of LaTeX can be credited to its proficient management of complex mathematical expressions and graphs. Notably, LaTeX's superiority lies in its capacity to incorporate external libraries, thereby enhancing its capabilities for document creation.

1.1 Motivation

With its proficiency in managing graphs, particularly with external libraries like TikZ, can be challenging for newcomers. The extensive documentation for TikZ might be overwhelming, and the learning curve can be time-consuming, especially for one-time or ad-hoc graph creation. To address these challenges, it would be beneficial for users to describe their goals using a clear and familiar syntax, such as JSON (JavaScript Object Notation), while adhering to a specified format. The JSON specification provided by users would then be used to generate the corresponding TikZ code. By adopting a more straightforward syntax, the aim is to make graph creation in LaTeX more accessible and efficient for users.

1.2 Scope

This project focuses on supporting the creation of flowcharts. Users will provide a JSON file following a predefined specification. This JSON file will include information about the graph, such as node IDs, node labels, edge information, edge labels, node types, and the desired orientation (LR or TB). The JSON specification considers properties like repetitive node labels,

unique node IDs, and a structure that allows for easy addition of future properties (e.g., color). Users can specify the orientation (LR or TB) in the JSON file, and the application will read this parameter to adjust the layout of the flowchart accordingly. The tool created in this project will generate a TeX file containing the corresponding TikZ code based on the information provided in the JSON file. Users will have the ability to visualise the generated TikZ code to ensure it aligns with their intended goal. In essence, this project comprises two primary components: TikZ code generation and visualisation.

A challenging aspect will be the visualisation of the graph, specifically ensuring that edges don't cross or minimising edge crossings to uphold a fundamental property of flowcharts—readability

1.3 Why Flowchart?

Flowcharts are common in various domains today, playing a crucial role in enhancing the comprehension of complex processes, workflows, and systems. They serve as a valuable tool for educators, enabling them to effortlessly create visual representations of processes to aid student learning. In technical fields like software development, engineering, and IT, professionals utilise flowcharts for tasks such as system design, troubleshooting, and process documentation.

Integrating flowcharts into documentation simplifies communication among team members and relevant parties. Making the implementation process easy encourages their inclusion in study materials, helping convey information effectively and *promoting better retention.*[8]

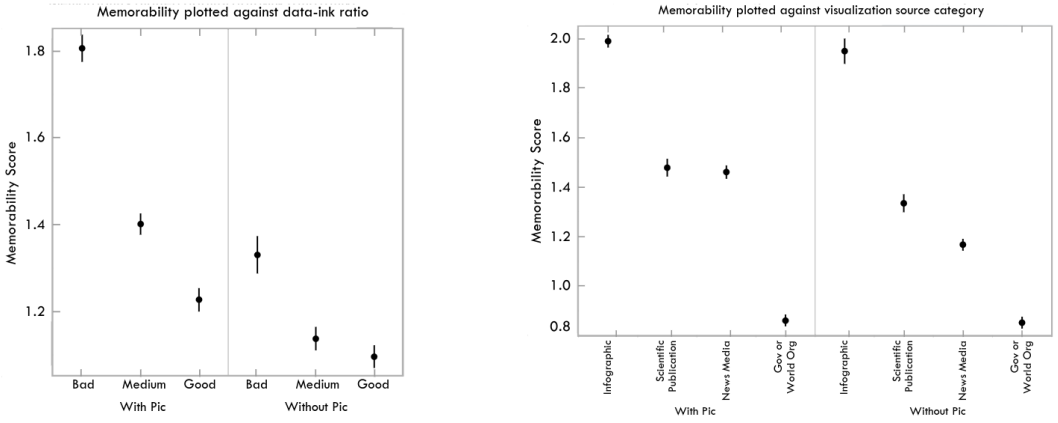


Figure 1.1: Study on what makes visualisation memorable [3, p. 8]

1.4 Objectives

The aim is to produce a standalone desktop application for latex users to successfully generate TikZ code and visualise the flowchart whose code has been generated from a JSON file. Below outlines the application's overview:

Input

The application should accept a JSON file following a specified structure containing data for flowchart generation.

Parsing

The application parses the content of the provided JSON file to extract relevant information for flowchart generation.

Output

It should enable users to generate a TeX file containing TikZ code based on the parsed data.

Visualisation

It should allow users to visualise the generated TikZ code, providing graphical representation of the intended flowchart. The application should align with common practices in flowchart visualisation.

User Interface

The interface should be easy to use following usability heuristics such as visibility of system status, match between the system and the real world, user control and freedom, consistency, flexibility, safety, efficiency (error handling), and aesthetics

Chapter 2

Literature

2.1 JSON (JavaScript Object Notation)

Data Interchange Format (DIF) is a standardised way of organising and representing data to facilitate communication between different computer systems or applications. However, one drawback of DIF is that it handles only single sheet within a workbook.[4]

JSON is rooted in a subset of the JavaScript Programming Language Standard ECMA-262 3rd Edition from December 1999. 'It is a lightweight data-interchange format'.[7] JSON is designed for human readability and ease of writing, as well as machine-friendly parsing and generation. While being entirely language-independent, JSON is constructed based on either a collection of key-value pairs or an ordered list of values, which are conventions familiar to programmers in the C-family of languages, namely C, C++, Java, JavaScript, Python, and more. These characteristics collectively position JSON as an ideal language for data interchange.

2.2 LaTeX & TikZ

Latex is a sophisticated typesetting tool; it excels in crafting high-quality documents. It is primarily used for writing scientific documents [see 1], but its versatility extends to various publishing. LaTeX allows users to concentrate on their content without being burdened by formatting complexities. Documents will follow the standard LaTeX structure. However, the document class and syntax for denoting the document's start and end will not be provided. This is because the generated TEX file in this project is intended to function as a subfile, which can be incorporated into a main document using the \input{filename.tex} syntax.[11]

On the other hand, Tikz (TikZ ist kein Zeichenprogramm, which translates to 'TikZ is not a drawing program' in German) is a package for creating graphics, especially diagrams (like flowcharts) and illustrations, in LaTeX documents, which are scalable. TikZ is based on the PGF (Portable Graphics Format) library, which provides a powerful and flexible framework for generating graphics[9]. To utilise TikZ in LaTeX, it is essential to include the preamble \usepackage{tikz}, along with the required libraries specified as \usetikzlibrary{library}. As previously mentioned, this project concentrates on flowcharts [see 1.2], calling for the use of the Sugiyama framework, which is a method used in graph drawing to organise and visualise directed acyclic graphs in hierarchical layout.[14] This is discussed further in the next section.

2.3 Visualisation (Graph Drawing)

For enhanced visualisation of flowcharts, several factors warrant consideration. These factors include minimising or avoiding edge crossings, avoiding node overlap, and ensuring that there is a flow characterised by a distinct source and direction. Numerous algorithms cater to these considerations, yet, given that flowcharts often denote hierarchy, employing hierarchical layout procedures emerges as particularly suitable. Among these, the Sugiyama method stands out as a well-suited choice. This method systematically organises graph nodes into layers or ranks and subsequently aligns these layers in a vertical arrangement. The algorithmic framework of the Sugiyama method comprises five key passes: Cycle Removal, Layer Assignment, Vertex Ordering, Coordinate Assignment, and Edge Construction.[13]

```
procedure draw_graph()
begin
    cycle_removal()
    node_ranking()
    node_ordering()
    node_positioning()
    edge_routing()
end
```

Figure 2.1: An overview of the Sugiyama method

2.3.1 Cycle Removal

The first step in the Sugiyama method is cycle removal, which aims to reverse some edges to transform the input graph into a Directed Acyclic Graph (DAG), essentially a graph without

directed cycles [1]. Common cycle removal algorithms include prioritised greedy cycle removal, greedy cycle removal, naive greedy cycle removal, and random greedy cycle removal.

Gansner et al. [6] proposed an alternative cycle removal algorithm utilising depth-first search, preserving the original flow of the input graph. This depth-first search (dfs) cycle removal assumes that the order of nodes and edges input by the user somehow reflects the user's intended structure, a notion aligned with the way individuals conceptualise their flowcharts.

The algorithm explores the graph using depth-first search, visiting vertices in the order of their creation. It consists of two parts: 'dfs cycle removal' which reverses back edges to eliminate directed cycles, and 'dfs visits' which keeps track of visited nodes.[10, p.58]

```
DFS-CYCLE-REMOVAL(G)
1    for each node v ∈ V(G) in the order of creation do
2        v.visited = false
3        v.stacked = false
4
5    for each node v ∈ V(G) in the order of creation do
6        DFS-VISIT(G, v)

DFS-VISIT(G, v)
1    if not v.visited then
2        v.visited = true
3        v.stacked = true
4
5        for each edge e ∈ OUTGOING-EDGES(G, v) in the order of creation do
6            u = GET-NEIGHBOR(G, e, v)
7
8            if u.stacked then
9                e.reversed = true // e is a back edge
10           else
11               DFS-VISIT(G, u)
12
13       v.stacked = false
```

Figure 2.2: Depth-First search cycle removal algorithm by reversing back edges [10, p.58]

2.3.2 Layer Assignment (Node Ranking)

In this phase, a directed acyclic graph is a prerequisite, necessitating the occurrence of the cycle removal phase. In the layering process, nodes are systematically organised into ranks, ensuring that edges exclusively connect nodes in adjacent ranks. The default layering algorithm is the minimum height layer assignment. An optimised alternative is the network simplex algorithm, designed to enhance layer assignment in a network. This is achieved through iterative adjustments to edges based on their cut values, promoting balanced and normalised layers. [10, p. 60] The cut value, an integer linked to each edge, plays a crucial role in this optimisation

```
LONGEST-PATH-LAYER-ASSIGNMENT(G)
1    V ← TOPOLOGICAL-ORDERING(G)
2
3    for each node v ∈ V do
4        v.layer = 1
5
6        for each edge e ∈ INCOMING-EDGES(G, v) do
7            u = GET-NEIGHBOR(G, e, v)
8            v.layer = MAX(v.layer, u.layer + e.minlength)
```

Figure 2.3: Longest path layer assignment algorithm [10, p.61]

process. It's noteworthy that, in practice, the network simplex assignment tends to execute faster with fewer iterations. However, it's important to acknowledge that its polynomial time solvability has not been proven.[6, p.217]

2.3.3 Vertex ordering (Node Ordering)

In this phase, the objective is to arrange nodes within each layer with a focus on minimising edge crossings. Two commonly employed vertex ordering methods are the Barycenter function and the Median function. Both methods operate on the premise that nodes should be positioned in close proximity to their neighbours to reduce edge crossings.

Gansner et al. enhanced the Median function which involves initially arranging nodes along the x-axis based on a crossing minimisation order. Subsequent iterations refine this layout through three distinct phases: Positioning, Straightening, and Packing. In the Positioning phase, methods like the median heuristic are employed to optimise node placement. To reduce

edge bends, the straightening phase attempts to align nodes along a path. While this may result in a wider drawing, a packing algorithm is subsequently applied to compress it horizontally without introducing new edge bends. [10, p.63]

2.3.4 Coordinate assignment (Node Positioning)

The algorithm for this step requires proper layering to be generated beforehand (see layer assignment). The objective is to minimise edge bends, ensure straight and compact edges, and avoid overlaps among nodes and edges. This involves setting x and y coordinates for nodes in two steps: assigning x coordinates based on rank order and y coordinates to nodes within the same rank. To optimise this, Gansner et al. applied the layer assignment problem to a graph with virtual nodes, demonstrating its correspondence to an optimal solution for the horizontal node position problem. [10, p.67]

2.3.5 Edge Construction (Edge Routing)

Edges in a diagram can be represented by either straight line segments, which often disregard avoidance of other segments and nodes, or by spline curves. Spline routing algorithms are typically divided into two sections: the upper section and the lower section.

In the upper section, a polygonal region within the layout is calculated to accommodate the spline. This section then calls upon the lower section to determine the optimal spline within that region. To finalise the process, the upper section adjusts the virtual nodes' sizes based on the spline's bounding box and confines the spline within the endpoint node shapes' boundaries by clipping it accordingly. [6, p.224]

The implementation of algorithms is briefly mentioned here, as it will be discussed in more detail in the implementation section [5]

Chapter 3

Requirements & Specifications

The goal of this project is to streamline the process of creating flowcharts in latex for users. The application must generate tikz code that represents what the user had in mind. Below are further discussions of the requirements and specifications of this project.

3.1 Requirements

Functional Requirements

The application must;

- RF1 - Allow the user to import a JSON file as an input.
- RF2 - Have a GUI for users to interact with.
- RF3 - Be able to read the appropriate JSON data containing key-value pairs and convert them into TikZ code.
- RF4 - Offer a visualisation feature for the generated TikZ code.
- RF5 - Display an error message and exit gracefully if it encounters an error.

Non-Functional Requirements

- RNF1 - The application must be able to replicate the same graph or TikZ code with the same input file.
- RNF2 - The interface should allow the user to edit the generated TikZ code.

- RNF3 - The application must include a functionality to export the generated TikZ code to a .tex file
- RNF4 - The application must support windows, macOS and linux operating systems.
- RNF5 - The generated TikZ code should be compatible with major LaTeX distributions like MikTeX and TeXLive

3.2 Specification

Requirement Code	Specification
RF1	Users should be able to provide the path to the JSON file they want to import as input.The application should verify that the provided path leads to a valid JSON file.
RF2	Develop GUI using TKinter for python. The GUI must have components like buttons, text fields and file selection diolog for user interaction
RF3	Users must ensure JSON input follows the defined structure [see 4.2]. Map keys in JSON data to the corresponding TikZ commands. This includes commands for drawing shapes, setting colors, and other visual properties.
RF4	A clearly labelled button for users to view generated code. Compile the generated TikZ code using pdflatex. Users can then preview the generated pdf.
RF5	Upon encountering an error, display the error message to the user in a message box. There should be guidance on potential resolutions or actions the user can take to address the error
RNF1	Use a data structure that maintains the order of elements
RNF2	Provide a text field where users can see the generated tikz code and edit it where necessary
RNF3	Add a save button to GUI for users to export generated code
RNF4	Application should be multi-platform. For example, use built-in os to get the path of a file instead of a string
RNF5	Tikz commands must adhere to the structure that is compatible with mikTex and TexLive distributions

Chapter 4

Design

In this chapter, an organised perspective on how different aspects of the project work together is presented. This will include an exploration of the project's technical foundation and an explanation of how users will interact with the system.

4.1 Development Environment

In the development of this project, the chosen operating system are windows 11 and macOS. The project is to be implemented using Python 3.12, a versatile programming language known for its simplicity and extensive libraries. Python's readability and ease of integration with graphical libraries makes it a suitable choice for this application.

4.2 JSON Specification

The aim here is to make the JSON structure easy to comprehend and concise. It includes global keys that affect the overall graph presentation. These keys include orientation, shape fill, shape colour, and font colour. Defining these keys and their corresponding values outside individual vertices ensures that they uniformly impact the entire graph.

Representing Vertices

A simple description of a vertex in the JSON specification consists of a key (the unique ID of the vertex) and a value (the properties of the vertex). The key is provided by the user, and the value includes properties such as label (the text displayed in the vertex), type (the vertex type, e.g., process, input/output, etc.), and next (an array of vertices connected to this vertex).

Other properties, such as font colour, shape fill, and shape colour, only affect this particular vertex.

```
1
2       "orientation": "dt",
3       "idx1": {
4           "label": "Start",
5           "type": "terminator",
6           "next": [
7               {
8                   "id": "idx2"
9               }
10          ],
11          "shapeFill": "red"
12      },
13      "idx2": {
14          "label": "Input",
15          "type": "io",
16          "next": [
17              {
18                  "id": "idx3"
19              }
20          ],
21          "shapeFill": "blue"
22      },
23      "idx3": {
24          "label": "Process 1",
25          "type": "process",
26          "next": [
27              {
28                  "id": "idx4"
29              }
30          ],
31          "shapeFill": "orange"
32      },
33      "idx4": {
34          "label": "Decision 1",
35          "type": "decision",
36          "shapeFill": "green"
37      },
38
39  }
```

Figure 4.1: Example content of a JSON file representing a graph

4.3 Class Diagram

Class diagrams provide a blueprint for the organisation of code, illustrating how classes interact, the attributes they possess, and the methods they execute. This section provides a model that visually represents the structure and relationships of classes in this project.

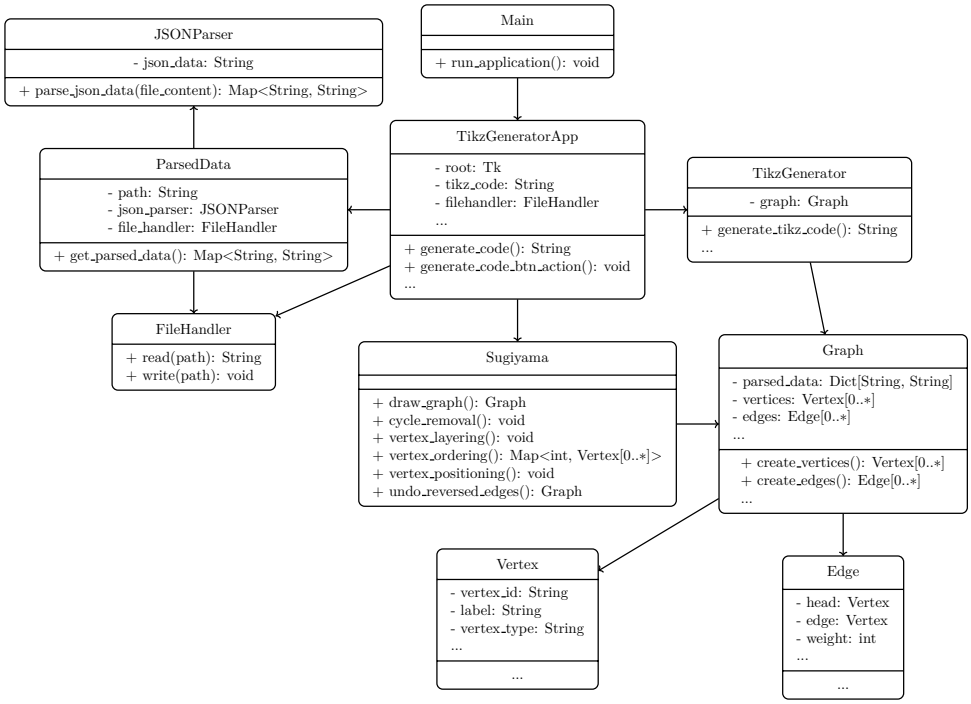


Figure 4.2: Class Diagram Illustrating System Components and Interactions

Main is where the user initiates the application.

TikzGeneratorApp is the main coordinating class that encapsulates instances of tkinter (for graphical user interface), TikzCodeGenerator, Filehandler, etc.

Graph is responsible for creating graphs based on the parsed json input. It holds all information about the graph including colour, orientation, vectex_type, etc. The class provides several methods to handle the graph creation process, including creating vertices and edges from parsed data.

Edge class represents an edge of the graph. It contains information about the vertices it connects, as well as additional attributes such as weight or label.

Vertex class represents a vertex of the graph. It holds information (for example, id, label,etc) about the vertex itself.

TikzGenerator generates TikZ code based on graph, which is created based on provided data. *generate_tikz_code()* method handles the mapping from details of the graph object to its corresponding TikZ code.

ParsedData class encapsulates information related to parsed data within the project. It contains attributes such as a file path (path), a JSON parser (json_parser), and a file handler (file_handler). The key method, get_parsed_data(), is responsible for retrieving and providing access to the parsed data by utilising the capabilities of the JSON parser and file handler. In

summary, this class centralises and manages the retrieval of parsed data.

JSONParser class is responsible for handling JSON data, with the file_content attribute representing the content of a JSON file. The method, parse_json_data(), takes the file content as input and performs the parsing of JSON data.

FileHandler handles file read and write operations.

Sugiyama method refines the graph to ensure that vertices have appropriate positions, thereby enhancing the readability of the graph.

4.4 Graphical User Interface (GUI)

The primary point of interaction between users and main applications is the graphical user interface. This section presents the design and functionality of the GUI to be implemented in this project, providing insights into its key features, user experience considerations, and the technologies employed.

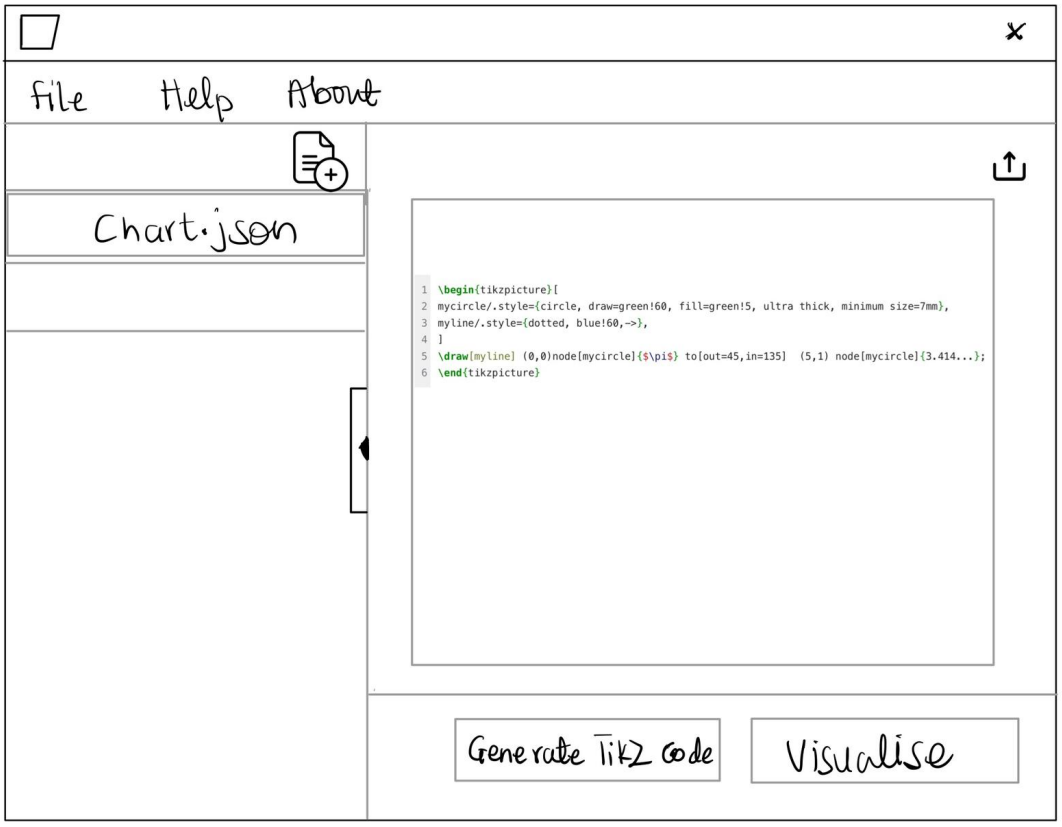


Figure 4.3: Image showing the GUI template

Key features

Import/Add files

After launching the application, if no file has been imported, and the user clicks on any of the buttons labeled 'Generate TikZ Code', 'Preview PDF', or 'Save', they will be prompted to add and select an appropriate file to proceed with the desired action.

Export/Save files

The user will be able to save a standalone .tex file, which can be included in another .tex file. After the user has generated TikZ code, clicking on the save button will allow the user to name the file and choose the destination.

Generate TikZ Code

The "Generate TikZ Code" button generates the corresponding TikZ code based on the imported or selected JSON file. If no file is imported or selected, the user will be prompted to do so before proceeding.

Preview PDF

This action assumes TikZ code has been generated prior. The user will be prompted to import the file and generate code to proceed with this action. By clicking the 'Preview PDF' button, the TikZ code is compiled, and a PDF is generated. The PDF, named 'tester.pdf' can be found in the root folder of the application for the user's preview.

User Experience Considerations

The layout of the GUI of this application is designed for user-friendliness by using clear and meaningful labels or familiar icons. It will also implement feedback mechanisms to ensure users are well-informed about the status of their actions. This includes error messages, tooltips, and text. In addition, the GUI will prioritise consistent design patterns, making it easier for users to recognise and learn.

4.5 Libraries And Other Applications in Use

TKinter

Tkinter is a standard Python library used for creating graphical user interfaces (GUIs), providing a simple and efficient way to build windows, dialogs, buttons, menus, and other interactive elements in Python applications. The GUI of this application will be implemented using the Tkinter library in Python, motivated by its simplicity and effectiveness.

TKinter Tooltip

TKinter-tooltip is an open-source Python library that allows you to add informative popups, known as tooltips, to widgets in Tkinter applications. These tooltips appear when the user hovers their mouse over a widget for a short while. The library offers customisation options like setting a delay before showing the tooltip, allowing the tooltip to follow the cursor's movement, and even using functions to dynamically generate the tooltip text.

PDFLatex (TexLive)

TeX Live is a comprehensive distribution of the TeX typesetting system that includes all the major TeX-related programs, macro packages, and fonts. It provides a complete TeX environment for various operating systems, including Windows, macOS, and Linux. TeX Live includes popular TeX engines like pdfLateX, XeTeX, and LuaTeX, along with a vast collection of packages. In this application, pdfLaTeX is used to compile the generated TikZ code, resulting in the creation of a PDF file, as mentioned earlier.4.4

l

Chapter 5

Implementation

While a class diagram was initially included in the application's design, its implementation follows an iterative approach. The project was planned to produce an improved working version at each milestone, and this plan also serves to ease tracking of the project's progress.

Key aspects considered in software development include loose coupling and high cohesion, ensuring that components perform singular tasks. This approach facilitates the addition of extra features, enhances maintainability, and promotes reusability.

5.1 MVP 1 - Basic Implementation

This foundational stage focused on establishing the core structure and functionality of the application. It involved implementing all classes within a single Python module. The primary goal was to generate functional TikZ code that accurately represents the provided JSON input. Additionally, this stage served as an experimental phase to explore different approaches for TikZ code generation. The classes implemented at this stage are as follows:

- **Filehandler**

  This class provides methods for reading and writing JSON files.The read method takes the path to a JSON file as input and returns the file's content as a string. If the file cannot be found or read, it raises a FileNotFoundError or IOError respectively.

  The write method writes the provided content (a string representing tikz code) to the specified path. If the file exists, the existing content will be overwritten, else, create a new file and add content. This class tackles requirements RF1, RNF3 and the reading aspect of RF2.

- **JSONParser**

  To obtain an appropriate data structure, the content of the JSON file, read using the file handle is passed as an argument. This is decoded into a dictionary of key-value pairs. If the content is unable to decode, a JSONDecodeError is raised. By parsing the file content, it enhances the mapping of a specific key to its corresponding tikz code.

- **ParsedData**

  This class orchestrates the extraction and formatting of JSON data to ensure it's ready for further processing. The structure is as shown above 4.2 however it results in a dictionary of type $Dict[string, Dict[string, string]]$ which is a suitable format for eventual use. It achieves this through interactions with FileHandler and JSONParser instances. Below are the key steps during the class' instantiation:

  - Path handling: accepts a file path as input upon creation and employs the $os.path.join$ method to ensure cross-platform compatibility in path construction. The formatted path is then stored in the $self.path$ instance variable for later use.
  - FileHandler interaction: instantiates a FileHandler object and stores it in $self.filehandler$
  - JSON parsing: leverages both self.path and $self.filehandler$ in conjunction with a JSONParser instance to extract and parse the JSON data from the specified file. The resulting parsed data is stored in $self.parsed_json_data$ for subsequent use.

- **TikzGenerator**

  At present, this class features a template method that returns a dictionary containing layout specifications for the TikZ code. The keys of this dictionary include, *preamble*, *start_tikz* and *end*, which respectively correspond to the Tikz code snippets $\backslash usetikzlibrary\{graphs, graphdrawing, positioning, quotes, shapes.geometric, arrows.meta\}$ and $\backslash usegdlibrary\{layered\}$, $\backslash begin\{tikzpicture\}$ and $\backslash end\{tikzpicture\}$. This approach ensures proper code structure and simplifies generation.

  The 'generate_tikz_code' method utilises this template and parsed input data to construct the TikZ code. It iterates through the parsed data and identifies relevant values based on specific keys within the data structure. The extracted values are then used to replace placeholders in a template, dynamically building the TikZ code string. Finally, the complete TikZ code string is returned which is later written to file.

  For example, in the node representation $f'node_id[as = label, node_type...]'$, placeholders such as '*node_id*', '*label*', and '*node_type*' are retrieved from the data using the appropriate

keys. If a key is missing, an empty string is returned as the value. The node representation is subsequently appended to the code using the syntax *'code += node_representation'*.

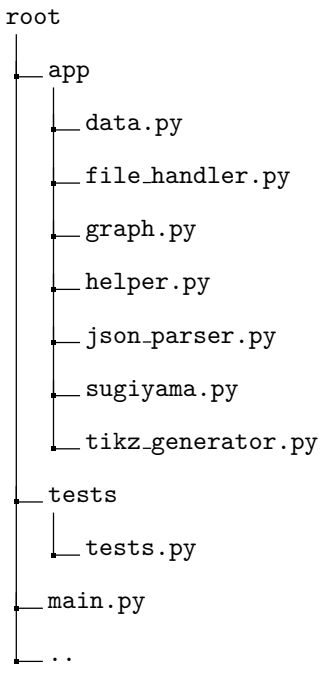
During this experimentation phase with TikZ generation, the JSON input was initially assumed to represent a Directed Acyclic Graph (DAG), and vertices were positioned manually. However, this manual positioning approach was short-lived due to the absence of strategies for reducing edge crossings, which led to suboptimal layouts.

To address this challenge, the decision was made to leverage the graph drawing library within TikZ, specifically employing the layered layout [14]. This library implements the Sugiyama framework [see 2.3], known for its effectiveness in hierarchical layouts.

To preview the generated TikZ code, a manual compilation process was undertaken using the 'Lualatex' engine, resulting in the generation of a PDF document.

5.2 MVP 2 - Refactoring and Code Improvement

It became evident at this stage that various classes in the application required refactoring into Python modules, as there was a lack of proper file structure. Adopting clean code practices and establishing a structured file layout not only promotes maintainability but also makes it easy to import modules into their respective components.



A figure depicting the file structure

Graph module

The graph module (graph.py) contains the classes including vertex, edge and graph.

The vertex class represents individual vertices (nodes) within the graph structure. It encapsulates various properties such as vertex ID, label, font color, and others specific to each vertex. To facilitate comparisons between vertices the class defines both equality '$__eq__()$' and inequality '$__ne__()$' methods. Additionally, it implements a hash method '$__hash__()$' to generate a unique hash value for each vertex object, which is useful for efficient data storage and retrieval in dictionaries or sets.

The edge class serves to represent a connection between two vertices. Upon instantiation, an edge object requires specification of the head vertex (vertex at the starting end of the edge - source), tail vertex (vertex at the ending end of the edge - destination), and edge weight (defaulting to 1 if unspecified - optional). It also maintains an instance variable named '$self.reversed$' (set to $False$ by default). This flag is used in later functions to determine if the direction of the edge has been reversed. Like the Vertex class, it defines methods for equality '$__eq__()$', inequality '$__ne__()$', and hash '$__hash__()$' to enable comparisons and efficient data management.

Below discusses the creation of graph based on parsed data using the vertex and edge classes.

- **Graph**

  The graph construction process begins with the creation of all vertices. These vertices are then linked together by creating edges. The graph object is instantiated with parsed data. A method named '$get_vertices_from_data(data)$' extracts the data without including global properties such as orientation. The resulting data is then used by another function named '$create_vertices$' to create the individual vertices within the graph. The '$create_vertices(data)$' method iterates through the parsed data in place and returns a dictionary containing vertex IDs and their corresponding vertex objects. For each vertex ID and its associated details, each detail is stored in a variable. These variables are then used to create a new vertex object. Finally, both the vertex ID and the newly created vertex object are added to a dictionary.

  Once the vertices exist, edges are created to link them together, using the '$create_edges$' method. The '$create_edges$' method commences by invoking the '$creat_vertices$' method to obtain a mapping of vertex IDs to their corresponding vertex objects. This information is stored within a variable named '$vertex_mapping$'. It then iterates through the provided data, extracting details regarding the subsequent vertices for each existing vertex. To es-

tablish each edge, the necessary vertices are retrieved from the '*vertex_mapping*' variable, ensuring accurate association. The created edges are stored in a list to maintain the order of creation.

In addition, the graph class has an instance variable '$self.ordering$' which is used in later functions to store the ordering of vertices based on layers. It defines other methods such as '$all_outgoing_edges(vertex)$', which returns all outgoing edges of a particular vertex, $all_incoming_edges(vertex)$, which returns all incoming edges of a particular vertex, as well as methods for creating dummy vertices and updating the graph.

5.3 MVP 3 - Solution Based on The Sugiyama framework

While the graphdrawing library offered advanced layout capabilities, it deviated from the project's focus on developing a solution for TikZ code generation. Moreover, it introduced complexities particularly regarding the control over the entire graph's positioning. To address this, a decision was made to not use the 'graphdrawing' library. Instead, below were the steps implemented to help achieve better graph layout:

Cycle Removal

The first step towards improved graph layouts involves converting the original directed graph into a directed acyclic graph (DAG), where it is not possible to follow directed edges and return to the starting vertex. This transformation is essential because it paves the way for the subsequent layering process. It ensures the graph has directional flow from source to sink. Algorithms 1 and 2 describe the implementation cycle removal function.

Algorithm 1: Depth-First search cycle removal algorithm by reversing back edges [10, p.58]

```
1 DFS-CYCLE-REMOVAL(Graph)
2 begin
3     visited ← {}
4     stacked ← {}
5     for each vertex v ∈ Graph.vertices in order of creation do
6         DFS-VISIT(Graph, v, visited, stacked)
```

- *Remarks on Algorithm 1*

  3 - 4: Initialise visited and stacked to an empty set. Visited set keeps track of vertices that have already been explored. The stacked set holds vertices in the current depth-first

Algorithm 2: Depth-First search visits to track visited vertices [10, p.58]

```
1  DFS-VISITS(Graph, vertex, visited, stacked)
2  begin
3      if vertex not in visited then
4          visited.add(vertex)
5          stacked.add(vertex)
6      for each edge e ∈ Graph.outgoing_edges(vertex) in order of creation do
7          if e.tail in stacked then
8              e.reversed ← true
9              swap e.tail and e.head /* i.e.  tail = head and head = tail      */
               else
10             DFS-VISITS(G, e.tail, visited, stacked)
11     stacked.remove(vertex)
```

search call stack. It helps the algorithm backtrack and continue the search in the next unvisited branch.

5 - 6: Invoke the DFS-VISIT function on each vertex in the graph. Each vertex becomes a vertex from which the depth-first search visits starts.

- *Remarks on Algorithm 2*

  3 - 5: This checks whether the current vertex has not been visited. If so, then mark as visited by adding it to the visited set. In addition, add the vertex to the stacked set.

  6 - 10: This iterates through all outgoing edges of the current vertex being explored. For each outgoing edge, it checks if the tail vertex (the vertex the edge points to) is present in the stacked set. The presence of the tail vertex in stacked indicates that a cycle has been detected. This is because the tail vertex is already part of the current DFS exploration path stored in stacked, and encountering it again suggests a loop. To break the cycle, two actions are performed (8-9). Firstly, mark the edge as reversed and then swap the head of the edge with its tail.

  If the tail is not the stacked set, then make a recursive call to the tail vertex of the edge.

  11: At this point, the current vertex has been processed, therefore remove from the stacked set.

Layer Assignment

The second step towards improved graph layouts involves a process called layering. Layering assigns vertical layers (or ranks) to the vertices in the graph. As previously mentioned, the layering step requires the input graph to be a directed acyclic graph because cyclic dependencies

in the graph can lead to inconsistencies in the layering process. Thus, different layering could be generated each time for the same graph. Consider a graph where vertex A is connected to vertex B, B to C, and C to A. When assigning layers to each vertex in the graph, we encounter three possible scenarios. A is assigned layer 1, B is assigned layer 2, and C is assigned layer 3. C is assigned layer 1, A is assigned layer 2, and B is assigned layer 3. B is assigned layer 1, C is assigned layer 2, and A is assigned layer 3.

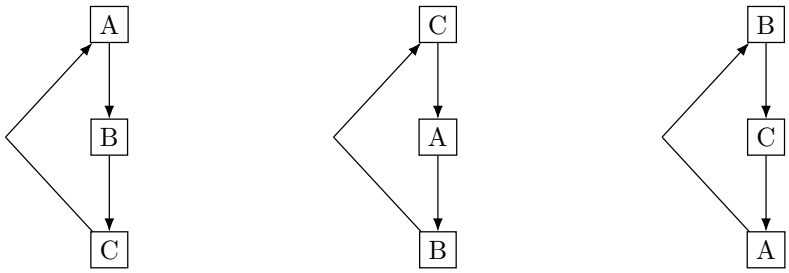


Figure 5.1: Showing different layering possibilities

This layering step provides a hierarchical structure necessary for subsequent ordering of vertices. Algorithms 3 and 4 describe the layer assignment implementation.

Algorithm 3: Topological ordering based on Kahn's algorithm [12]

```
1  TOPOLOGICAL-ORDERING(Graph)
2  begin
3      sorted_vertices ← empty list
4      source_vertices ← list of all source vertices of Graph
5      marked ← {}
6      while source_vertices is not empty do
7          vertex ← source_vertices.pop()
8          append vertex to sort_vertices
9          for edge e ∈ Graph.outgoing_edges(vertex) do
10             if e is not in marked then
11                 add e to marked
12                 if all incoming edges of e.tail is in marked then
13                     append edge.tail to source_vertices
14     return sorted_vertices
```

- *Remarks on Algorithm 3*

  3 - 5: Initialise the variables *sorted_vertices*, *source_vertices* and *marked* to an empty list, list of vertices with no incoming edges and empty set respectively. The '*marked*' variable keeps track of edges to avoid revisiting.

  6 - 13: This repeats if there are vertices in the source vertices list. If there exist a vertex in this list, one vertex is removed(popped) from the end of the list and appended to the

Algorithm 4: Longest-path layer assignment [10, p.61]

```
1 LONGEST-PATH-LAYER-ASSIGNMENT(Graph)
2 begin
3     vertices ← TOPOLOGICAL-ORDERING(Graph)
4     for vertex in vertices do
5         vertex.layer ← 1
6         for edge in Graph.incoming_edges(vertex) do
7             vertex.layer ← maxof(vertex.layer, edge.head.layer+1)
8     return Graph
```

list of sort vertices. The algorithm then iterates through a list of outgoing edges from the popped vertex. For each edge, if it is not marked, it is added to the list of marked edges (9-11). If all incoming edges of the tail vertex of the edge is marked then append the tail vertex to the list of source vertices (12-13).

14: Return the list of sorted vertices.

- *Remarks on Algorithm 4*

  3: Initialise the *vertices* variable to list of sorted Vertices by invoking the topological sort function.

  4 - 7: For each vertex in the '*vertices*' variable, their layer is set to 1. Then for each edge in the list of incoming edges of the vertex, the layer is assigned the maximum value between the current layer value of the vertex and the layer value of the head vertex of the incoming edge, incremented by 1.

  8: Return the graph with layer assigned to vertices.

Vertex Ordering

This step combines the introduction of dummy vertices and initial ordering to help achieve an improved graph layout.

The vertex ordering step requires the input graph to have a proper layering [10, p.59]. Meaning the absolute difference between the layer value of a vertex and their outgoing or incoming vertex is 1. Proper layering ensures edges are short and helps to reduce edge crossing leading to a clearer visualisation. To achieve proper layering dummy vertices are placed between edges whose layer difference is greater than 1. Further discussion of this technique is shown later in this section [refer to 9].

- *Remarks on Algorithm 5*

  2 - 3: Initialise the variables *dummies* and *edge_require_dummies* to empty lists. The

Algorithm 5: Adding dummy vertices [13, p.419]

```
   ADD-DUMMY-VERTICES(Graph)
1  begin
2      dummies ← empty list
3      edge_require_dummies ← empty list
4      for edge ∈ Graph.edges do
5          layer_diff ← edge.tail.layer - edge.head.layer
6          if layer_diff > 1 then
7              add dummy vertices to bridge the gap in layer /* i.e layer_diff-1 dummy
                   vertices required to bridge the layer gap                    */
8      for vertex v ∈ dummies do
           add edges to maintain connectivity
9      update Graph
```

dummies variable holds a list of lists representing dummy vertices for each "improper" edge while *edge_require_dummies* holds a list of copies edges that are not "proper".

4 - 7: This iterates through all the edges of the graph. For each edge, it calculates the difference in layer values between the head vertex (source vertex) and the tail vertex (destination vertex). If the layer difference is greater than 1, add dummy vertices to link the difference between the layers. In the implementation of edges, they are only formed between two vertices (head and tail). To place a dummy vertex between an edge, either the head or tail must be replaced and then reconnected by creating a new edge. For example, if an edge has $layer_diff$ of 2, the maximum dummies required to achieve proper layering is $layer_diff$-1. To insert the dummy vertex for this edge, the new dummy vertex is created, the original edge is copied and appended to the *edge_require_dummies* list. The head of the edge is stored in a temporary variable and the dummy becomes the head of the edge. The dummy vertex is marked as an end vertex and appended to a list. If the edge requires more dummies vertices, additional dummy vertices are created and appended to the same list. The original head vertex of the edge is finally appended to the list. The entire list is then appended to the dummies list.[refer to figure 5.2]

8: Once dummy vertices have been created for edges with large layer differences, this step establishes connections between them. The dummy vertices are connected by iterating through the dummies list, which contains sub-lists of vertices. For each of the vertices, starting at the second element(with index 1), create a new edge where the head is the current vertex and tail is the previous vertex. Get the matching head from the *edge_require_dummies* list and use its value for "*reversed*" to create the edge. In this

way, consistency in the direction and the original property of the edge is maintained. For instance, if the edge to which we are adding dummy vertices is reversed, then the bridging edges are created using the properties of the reversed edge. Consequently, the bridging edges are also labeled as reversed.

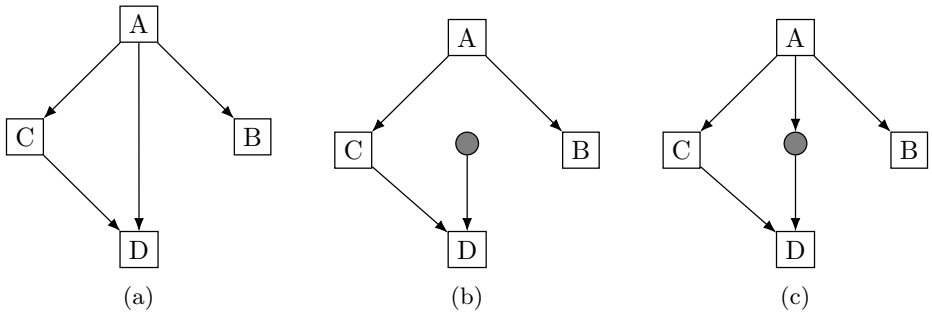


Figure 5.2: Showing the addition of a dummy vertex

Algorithm 6: Initialise ordering [6, p.220]

```
1  INIT-ORDERING(Graph)
2  begin
3      start_vertex ← any source vertex
4      ordered_vertices ← mapping of key-value pairs where key is the layer and value the
         list of vertices on the layer
5      DFS(vertex v)
6      begin
7          append v to the list of its respective layer
8          remove duplicates from list
9          for edge e ∈ Graph.outgoing_edges(v) do
10             DFS(e.tail)
11     DFS(start_vertex)
12     update Graph's ordering
```

- *Remarks on Algorithm 6*

  3 - 4: The algorithm starts by selecting a source vertex and assigning it to a variable named '*start_vertex*'. The '*ordered_vertices*' variable is initialised with keys representing layer values (e.g., 1, 2, 3, etc.). Each key is initially mapped to an empty list.

  5 - 12: This part describes a depth-first search(DFS) exploration. The current vertex is appended to the list of its respective layer. For each edge, the DFS function recursively explores the vertex at the tail end of the edge. This recursive exploration continues until all reachable vertices from the starting vertex have been visited.

Cross Reduction

After vertices have been ordered, a crucial step is to reorder the vertices in each layer to reduce edge crossings. This reduction can be achieved in two phases namely, layer cross minimisation and sift.

Layer cross minimisation is based on a heuristic of "adjacent delta" [2]. Adjacency delta represents the difference between current vertex's position and the average position of its adjacent vertices in the ordering. A lower adjacent delta signifies a closer alignment between the current vertex's position and the average position of its neighbours (tail vertices of the current vertex's outgoing edges). This is desirable because edges connecting such vertices are less likely to cross other edges in the layout.Conversely, a high adjacency delta indicates further alignment between the vertex's position and its neighbors' average position. This suggests a higher chance of edge crossings for edges connected to this vertex.

Sifting on the other hand, goes through vertices on the same layer and swap them to comparing the number of crossings at each swap. If the swap leads to a decrease in the total number of edge crossings, the swap is permanent, and the process moves on to the next vertex in the layer.If the swap does not improve the crossing count, the vertices are swapped back to their original positions, and the algorithm considers swapping the current vertex with the next neighbour.[2]

Algorithm 7: Layer cross minimisation [2]

```
   LAYER-CROSS-MINIMISATION(Graph)
 1 begin
 2     ordering ← Graph's current ordering
 3     not_matching_orders ← true
 4     while not_matching_orders do
 5         ordering_copy ← copy of ordering
 6         for layer = 1 to maximum layer do
 7             adjacent_delta_list ← empty list
 8             for each vertex on layer do
 9                 average_tail_position ← sum of tail vertex positions / total number of
                     tail vertices
10                 adjacent_delta ← abs(current vertex's position - average_tail_position)
11                 append a tuple of (vertex, adjacent_delta) to the adjacent delta list
12             sort the adjacent delta list in increasing order based on adjacent delta
13             update the ordering of the layer
14         if ordering == ordering_copy then
15             not_matching_orders ← false
```

- *Remarks on Algorithm 7*

2 - 3: The layer cross minimisation algorithm starts by initialising a variable named '*ordering*' to the input graph's current ordering. Another variable '*not_matching_orders*', which acts as a check to determine if the ordering of vertices has changed during the optimisation process, is set to true.

4 - 11: This continues until there are no further improvements in the vertex ordering. A copy of the current ordering is stored in a variable '*ordering_copy*', which serves as a reference point for eventual comparison. Once a copy of the current ordering is stored, the algorithm goes through each layer and calculates the "adjacent delta" for each vertex on a particular layer. This is achieved by getting the indices (positions) of all the tail vertices (destination vertices) connected to the current vertex through outgoing edges then calculate the sum. The sum is divided by the total number of outgoing edges for the current vertex. This average is deducted from the index(position) of the current vertex.

12 - 15: Each adjacent delta is stored together with its vertex as a tuple(ordered pairs) in a list and sorted based on the adjacent delta in increasing order. Sorting by adjacency delta prioritises vertices that are currently positioned far from their neighbors (high delta) over those well-aligned with their neighbours (low delta). The current layer's ordering within the overall graph ordering is updated with the new order derived from the sorted list. If the ordering of the graph is the same as the copy made earlier, then the graph cannot be optimised any further so the '*not_matching_orders*' is set to false to exit the loop.

- *Remarks on Algorithm 8*

  2 - 7: The algorithm initialises the "*max_iterations*" variable to 10. The reference [2], suggests that 10 iterations are generally sufficient to achieve a significant reduction in edge crossings. The graph's current ordering is also stored in a variable named *ordering*. Then sort all vertices based on their degrees in decreasing order and store it in a variable Q. This is because prioritising vertices with higher degree generally yields better results (i.e lesser edge crossings)

  7 - 18: For each iteration, traverse vertices in the order of Q. For each vertex, get sibling vertices on the same layer. Swap the positions of each sibling vertex with the current vertex. Count the number of crossings above and below the vertex caused by its incoming and outgoing edges. Append the number of crossings to with the entire list as a tuple to the list named "best". Once all siblings have been visited, the layer of the vertex in the ordering is then updated with the list with the least number of crossings.

Algorithm 8: Sifting [2]

```
   SIFTING(Graph)
 1 begin
 2     max_iterations ← 10
 3     ordering ← Graph's current ordering
 4     vertices_degree ← empty list
 5     total_degree ← in_degree + out_degree
 6     append a tuple of (vertex, total_degree) to the vertices_degree list
 7     Q ← sorted vertices based on degree in decreasing order
 8     for i=0 to max_iterations do
 9         for vertex in Q do
10             best ← empty list
11             for sibling_vertex in vertices of same layer as vertex do
12                 crossing ← 0
13                 swap the positions of sibling_vertex and vertex
14                 crossing += count_crossing_above_vertex(vertex)
15                 crossing += count_crossing_below_vertex(vertex)
16                 append a tuple of (copy of vertices of same layer, degree) to best
17                 swap the positions of sibling_vertex and vertex back
18             update the vertex's layer of the ordering to the list with the least 'crossing'
                in the 'best' list
```

The cross-counting methods uses a brute force approach to count the number of crossing of a specific edge. To identify crossings for a specific edge, the algorithm considers all incoming edges to the current vertex's layer. For each incoming edge, it compares the indices of the head and tail vertices. If the current edge's tail vertex index is higher than the incoming edge's tail vertex index, and the current edge's head vertex index is lower than the incoming edge's head vertex index, then a crossing is detected.

Vertex Positioning

This stage involves assigning final positions to the vertices within the graph. The algorithm employed uses an iterative process that sweeps up and down the layers of the graph, adjusting the positions of vertices on each layer to optimize the overall layout.

- *Remarks on Algorithm 9*

  3: Initialise the variable '*vertices_x_position*' as a mapping of each vertex to the position 0.

  4 - 6: The functions '$DOWN - SWEEP$' and '$UP - SWEEP$' perform a downward and upward sweep, adjusting the values of '*vertices_x_position*' for each vertex. This is repeated twice within the loop.

  7 - 9: For each vertex and x-coordinate in 'vertices_x_position', assign the vertex's position

Algorithm 9: coordinate assignment [15]

```
1 COORDINATE-ASSIGNMENT(Graph)
2 begin
3     vertices_x_position ← map of each vertex as key mapping to 0
4     for i=0 to 2 do
5         DOWN-SWEEP(Graph, vertices_x_position )
6         UP-SWEEP(Graph, vertices_x_position )
7     for vertex, x in vertices_x_position do
8         vertex.position ← (x, vertex.layer*4)
9     return Graph
```

to the x-coordinate and scale its layer by 4. The choice of 4 as a scaling constant resulted from experimentation, as it tends to work well with most graphs.

Algorithm 10: Down sweep [15]

```
1 DOWN-SWEEP(Graph, vertices_position)
2 begin
3     for layer=1 to max_layer do
4         layer_vertices ← get the vertices on the layer
5         for vertex in layer_vertices do
6             incoming_positions ← positions of incoming vertices from 'vertices_position'
7             position ← MEDIAN-POSITION(incoming_positions)
8             vertices_position ← ADJUST-VERTEX-POSITION(vertex, position,
                vertices_position, ordering)
```

- *Remarks on Algorithm 10*

  5 - 8: For each vertex on a particular layer, retrieve the already adjusted positions of incoming vertices. These incoming positions are then used to calculate the median position. The function 'MEDIAN-POSITION' calculates the median position by selecting the median value from the list of incoming positions. If the size of the list is even, the function chooses both the middle index and the index before it. It then adds the values at both indexes and divides by 2 to obtain the median position.

  The UP-SWEEP algorithm is similar to the DOWN-SWEEP. However, it begins from the maximum layer and iterates towards the least layer. Additionally, it retrieves the already adjusted positions of outgoing vertices.

- *Remarks on Algorithm 11*

  4 - 12: After identifying vertices on the same layer as the input vertex, the algorithm finds the leftmost and rightmost vertices already positioned relative to the input vertex.

Algorithm 11: Adjust vertex position [15]

```
1  ADJUST-VERTEX-POSITION(vertex, position, vertices_position, ordering)
2  begin
3      vertices_on_layer ← vertices on the input vertex's layer
4      leftmost ← None
5      rightmost ← None
6      leftmost_list ← vertices already positioned to the left of the current vertex
7      if leftmost_list is not empty then
8          leftmost = maxof(leftmost_list) /* vertex with the greatest position  */
10     rightmost_list ← vertices already positioned to the right of the current vertex
11     if rightmost_list is not empty then
12         rightmost = maxof(rightmost_list) /* vertex with the least position   */
13     vertex_width ← 4
14     vertex_offset ← 4
15     min_x ← leftmost position + vertex_width + vertex_offset
16     max_x ← rightmost position - vertex_width
17     if position < min_x then
18         position ← min_x
19     else if position > max_x then
20         position ← max_x
21     return position
```

If there are vertices positioned to the left, it selects the one with the greatest position as the leftmost. If there are vertices positioned to the right, it selects the one with the least position as the rightmost.

13 - 21: This part of the algorithm calculates minimum and maximum allowed positions for the input vertex based on the positions of neighbouring vertices, considering the predefined vertex width and offset. The vertex width is set to 4 units, which provides adequate spacing considering that the minimum width of a vertex in the flowchart is 3 units. If the desired position is outside the allowable range, it adjusts it to the nearest valid position within the range. And finally returns the adjusted position.

Finalising the Graph

Once the input graph has undergone the steps discussed above, one of the final tasks is to undo reversed edges. The UNDO-REVERSED-EDGES function simply iterates through all edges, and if the edge is marked as reversed, it swaps the head vertex with the tail vertex.

The last step involves setting the orientation of the graph. The default orientation is from top to bottom. Each of the remaining orientations has a corresponding function that implements it. For example, the ”lr” (left-to-right) orientation uses the ‘LEFT-TO-RIGHT’ function. This

function retrieves the positions of all vertices and applies a 90-degree clockwise rotation to them.

At this milestone, the application ran in the terminal. The generated TikZ code is written to a file, and a bash script is executed to compile the TikZ code and open the resulting PDF. The following section discusses the integration of the graphical user interface.

5.4 MVP 4 - Graphical User Interface (GUI) Integration

To achieve separation of concerns, the module "tikz_generator_app.py" is created to solely implement the GUI functionalities. This module contains a class named "TikzCodeGeneratorApp," which takes an instance of tkinter as an instance variable.

The tkinter classes utilised in this application include Menu, Button, Radiobutton, PhotoImage, etc. Instances of these classes were employed in methods such as "create_menubar" to generate the menu bar and menu of the application, and "create_toolbar" to establish the toolbar.

The key features of the GUI include adding a JSON file, generating TikZ code, previewing the generated TikZ code, and exporting the generated code to a .tex file. Below illustrates how each feature was implemented.

Adding JSON Files

To implement the "add file" action, a button is created through the "create_toolbar" method. The action of this button is handled by the "file_open_actio" method. This method utilises the file dialog in tkinter and invokes the "askopenfilename" method. The dialog includes parameters such as a title, initial path, and filetypes. Specifically, the file types are restricted to ".json" files.

Upon selecting a file, it appears on a list of files, and any existing radio button selections are reset. Additionally, duplicates are removed from the list of file paths.

Generating TiKZ Code

The action performed upon clicking the "generate TikZ code" button is implemented in the "generate_code_action" method. This method first checks if there is a valid file path. If a valid path exists, it calls the "draw_graph" function in the sugiyama.py module. The resulting graph is then passed as input to an instance of the TikzCodeGenerator class.

The "generate_tikz_code" method is then invoked on the TikzCodeGenerator instance, and the generated code is displayed in the text box.

Previewing Generated Code

The generated TikZ code is written to a file named "tikz.tex" using the write method in the file handler class. Python's "subprocess" module is employed to execute a script responsible for compiling the main .tex file named "tester.tex".

In the main .tex file, the "$\backslash input$" syntax is used to include the "tikz.tex" file, integrating the generated TikZ code into the final document.

Exporting TiKZ Code

The primary action method for saving or exporting files is the "file_save_action". This method first verifies that the text box is not empty. It then uses the "write" method in the file handler class to save the generated code to a file. The file dialog module in tkinter is employed to set the destination of the exported file.

Additional Features

Copying Generated Code

The copy action method uses the pyperclip package to copy the content of text box to the operating system's clipboard.

Clearing Text Area and Deleting Files

The graphical user interface also includes a button for clearing the text area. This is achieved by calling the delete method on the Textbox instance and using the parameters "1.0" and "tk.end".

Additionally, there is a button to delete files. This is accomplished by removing the particular file path from the list of file paths, clearing the radio button, and then rebuilding it.

Chapter 6

Legal, Social, Ethical and Professional Issues

The Code of Good Practice issued by the British Computer Society covers various ethical guidelines and standards that guide professionals in the field of computer science. Thus, computer scientists are expected to adhere to the Code of Good Practice, which is also applicable to this project.

The user interface of this project incorporates commonly used icons found in other applications, which helps ease of learning for users. This use of familiar design conventions aligns with the principle of usability, particularly emphasising visibility and feedback. The interface employs tooltips and message boxes to provide timely and relevant feedback to users, which enhances their overall experience.

Regarding the development process, this project consists of original code along with references to research resources, Python libraries and Tex. Providing clear reference to resources upholds the fundamental principle of protecting intellectual property. It is very important to ethically utilise resources and ensure their proper referencing and authorisation, as failure to do so can undermine the credibility of the original authors. Extensive effort has been devoted to providing accurate and comprehensive references to all used resources within this project.

In terms of security and privacy considerations, the application does not retain any user information. It operates solely on the user's local operating system, which leaves users greater control over their personal and sensitive data. Doing so mitigates potential risks associated with unauthorised access to personal information. This approach indicates a commitment to safeguarding user privacy and security.

Chapter 7

Evaluation & Testing

7.1 Evaluation of algorithms

In the evaluation of the algorithms used in this project, correctness of the algorithm, the concept of asymptotic runtime and comparison with other algorithm that achieves the same goal.

An algorithm's correctness can be defined in two ways: logical and functional correctness. Logical correctness is about ensuring that the algorithm is logically sound and free from any flaws in its logical structure. Functional correctness on the other hand, focuses on whether the algorithm produces the correct output for a given input according to its specification.

Asymptotic runtime of algorithms used in this project refers to how the execution time of the algorithm changes as the size of the input gets very large (worst case). The Big O notation is the most common way to express asymptotic runtime. It describes the upper bound on an algorithm's growth rate. This notation is used for both time complexity and space complexity. Space complexity, like time complexity, describes how the memory usage of an algorithm grows as the size of the input grows.

Cycle Removal

It has been shown that depth first search (DFS) is a valid algorithm for graph traversal and can be used to detect cycles [6, p.216]. By maintaining a "stacked" set to keep track of vertices in the current DFS call stack, the function can detect cycles. If during the traversal, a vertex is encountered that is already in the stacked set, it indicates the presence of a cycle. This is confirmed by reversing the direction of the edges in the cycle.

The depth-first search cycle removal algorithm implemented in this project has a time

complexity of $O(|V| + |E|)$ where $|V|$ is the number of vertices and $|E|$ is the number of edges. This is because the algorithm explores each vertex and edge exactly once.

The space complexity however is $O(|V|)$ due to the space used by the "visited" and "stacked" sets. These sets store vertices that have been visited and vertices currently in the DFS call stack [refer to 5.3]. In the worst case, the algorithm might visit all vertices and need to store them in these sets, leading to $O(|V|)$ space complexity.

In comparison with the priority greedy cycle removal algorithm [10, p.55], the enhanced greedy algorithm prioritises sources, sinks, and vertices with a maximum difference of out and indegree, whereas DFS doesn't prioritise vertices based on specific characteristics. DFS simply explores vertices based on the order of traversal.

The runtime of the enhanced greedy algorithm is linear in the size of the input graph [10, p.55]. This means that the time complexity of the algorithm is directly proportional to the number of vertices ($|V|$) and edges ($|E|$) in the graph thus, time complexity is $O(|V| + |E|)$ which is also the case for DFS.

The effectiveness of each algorithm depends on the characteristics of the input graph. The enhanced greedy algorithm may perform well on graphs with certain properties, such as having identifiable sinks, sources, or vertices with significant out-in degree differences. For example, having few cycles in the input graph allows the algorithm to systematically reduce the complexity of the graph and efficiently remove cycles. DFS, on the other hand, is a more general approach that works well for various types of graphs. Also, by reversing edges in the graph as part of a DFS cycle removal process, the algorithm is essentially changing the direction of the edges while preserving the underlying adjacency information.

Layer Assignment

The layer assignment algorithm uses Kahn's algorithm to perform topological ordering, ensuring that it correctly orders vertices in a directed acyclic graph (DAG). This part of the algorithm is proven correct as Kahn's algorithm is a well-established method for topological sorting. It intuitively visits source vertices and places them in a list different list which effectively sorts the vertices topologically. This is known because the input input graph is a DAG and there will be at least one vertex with no incoming edges.

The correctness of the longest path layer assignment algorithm can be proven by mathematical induction. Considering the smallest possible input graph, like an empty graph or a graph with just a single vertex. The algorithm holds in each of these cases and it is trivial in the sense

that there are no edges.

In the induction hypothesis step, it is assumed that the algorithm correctly assigns layers to n vertices in a graph.

To prove that the algorithm holds for $n + 1$ vertices in a graph, it is known by induction hypothesis that the first n vertices in the topological order assigns layers correctly. By the topological sorting property, all the incoming vertices of the $(n + 1)th$ vertex is already in n. The algorithm computes the layer of the $(n+1)th$ vertex by considering the maximum layer of its incoming vertices plus one, ensuring that it is correctly assigned the layer. Therefore, since the algorithm correctly assigns layers to vertices for a graph with n vertices and it also correctly assigns layers for a graph with $n + 1$ vertices, it demonstrates it correctly assigns layers for all graphs by mathematical induction.

The time complexity of this algorithm is $O(|V| + |E|)$ in the size of the input graph. This is because each vertex and edge is visited exactly once.

The space complexity, however, is $O(|V|)$ with respect to the number of vertices in the input graph. The is because the algorithm visits each vertex and stores the visited vertices in order in a list.

Another approach to the layer assignment problem is the network simplex layer assignment. It constructs an initial feasible tree and iteratively replaces edges with negative cut values, followed by normalisation and balancing of layers [6, p.218].

In comparison, the longest path layer assignment algorithm focuses on finding the longest paths in the graph, which might not necessarily result in balanced layers while the network simplex layer assignment algorithm prioritises optimising a specific criterion, like cut values and tree structures, which might lead to a different layer assignment strategy compared to simply maximising path lengths. Also, the network simplex layer assignment algorithm generally leads to the graph requiring fewer dummy vertices to achieve "proper" layering [refer to 11]. Although, the network simplex layer assignment algorithm strives for optimal layer assignment, it comes with some tradeoffs. Its time complexity is $O(|VE|)$ in the size of the input graph.

Vertex Ordering

- **Addition of Dummy Vertices**

  For the correctness of this algorithm, preservation of original vertices and edges, proper layering and addition of vertices, are properties of the output graph.

  Although the algorithm goes through each edge and modifies them to add dummy vertices,

it does not remove or modify the properties of existing vertices or change the direction of existing edges. Therefore, all original vertices and edges remain unchanged in the output graph.

The algorithm ensures that the difference in layers is bridged by adding dummy vertices, resulting in a span of 1 layer for each edge. It adds dummy vertices only where needed to bridge gaps in layers, without affecting the connectivity or properties of the original graph. This demonstrates that the algorithm behaves correctly for all possible inputs, ensuring that the output graph maintains the desired properties.

The main operations of this algorithm include iterating through all edges of the graph, which has a time complexity of O($|E|$) where $|E|$ is the number of edges. For each edge, the difference between layers is calculated, which takes constant time (O(1)) because it involves simple arithmetic operations. Additionally, inserting a variable number of dummy vertices based on the difference in layers takes constant time because it involves a loop from 2 to maximum layer difference. This is effectively constant time as it's bounded by a small constant. Overall, the dominant factor in the time complexity is the loop through the edges, resulting in an final time complexity of O($|E|$).

- **Vertex Ordering**

  The correctness of the "init ordering" algorithm be can shown by verifying that it satisfies expected outcomes where vertices are grouped by their layers in the "ordered_vertices" dictionary and, within each layer, vertices are ordered based on their traversal sequence during DFS.

  As described in the implementation section [refer to 8], the algorithm initiates DFS traversal from a specified or the first source vertex if none is provided, and orders vertices based on their traversal sequence within layers. The resulting "ordered_vertices" dictionary stores vertices grouped by layer and ordered within each layer, which meets the expected outcomes for initialising vertex ordering within layers.

  Each vertex and edge in the input graph is visited exactly once in the worst case during the DFS traversal therefore the time complexity of the DFS traversal is O($|V| + |E|$). The algorithm also updates the ordering of vertices within layer which involves appending vertices to the "ordered_vertices" dictionary. This operation occurs at constant time (O(1)). Therefore the overall time complexity is O($|V| + |E|$) where V is the number of vertices and E is the number of edges in the graph because it remains the dominant factor.

Cross Reduction

To show the correctness, verification criteria include reduction in edge crossings and ensuring that the optimisation does not disrupt the connectivity of the graph. This was shown through manual testing with multiple graphs [refer to 7.2].

To establish the time complexity of the layer cross minimisation, the depth of iteration and iteration operations are considered. This phase iterates until the ordering converges, which can occur in a few iterations or may take more iterations in complex graphs. So, the maximum number of iterations can be denoted by I. The iteration operations, that is, calculating the adjacent delta by iterating over its outgoing edges and computing the average position of adjacent vertices in each layer and sorting them depends on the number of vertices and edges in the graph. Thus, the runtime of each iteration is $\mathrm{O}(|V| \times |E|)$. Since the maximum number of iterations is bounded by I, the overall time complexity can be approximated as $\mathrm{O}(|I| \times |V| \times |E|)$.

For space complexity, let N be the space complexity of storing a copy the ordering, and T be the space complexity of temporary variables to hold vertices and their respective adjacent delta. Therefore, the space required to store these variables can be approximated as $\mathrm{O}(|N| + |T|)$.

The time complexity of "sifting" depends on various parts of the algorithm including calculation of vertex degree which has a time complexity of $\mathrm{O}(|V| + |E|)$. This is because the algorithm iterates through all vertices in the graph and counts the number of outgoing and incoming edges of each vertex.

The vertices are then sorted based on their degrees which takes $\mathrm{O}(V \log V)$ [5]. For each vertex, the algorithm iterates over vertices on the same layer ($\mathrm{O}(L)$) and their outgoing edges ($\mathrm{O}(U)$). The counting of crossings contributes $\mathrm{O}(|S| \times |L| \times |U|)$ to the overall time complexity, where S is the average number of incoming vertices. This is because iterating through incoming edges of a vertex takes $\mathrm{O}(S)$. The algorithm then uses an inner loop to iterates through vertices on the same layer for each head of the incoming edge. The layer vertices are represented by L, which takes $\mathrm{O}(|L|)$. The algorithm goes through outgoing edges (U) from each vertex in the previous inner loop which takes $\mathrm{O}(|L|)$. Therefore the final time complexity of the sifting algorithm is $\mathrm{O}(V(1 + \log |V|) + (|S| \times |L| \times |U|) + |E|)$.

The algorithm uses data structures to store the vertex ordering, vertex degrees, and temporary variables during iterations which depends on the size of the graph and maximum number of iterations. We can let N represent storage for ordering, D for vertex degrees and T for temporary variables. The overall space complexity can be approximated as $\mathrm{O}(N + D + T)$.

Coordinate Assignment

This algorithm is shown to be correct to some extent from the results of the output graph during testing. There is a clear indication that some part of the entire has been centred and correctly position vertices, avoiding overlaps. This is further discussed in the testing section [refer to 7.2].

The time complexity of the algorithm can be deduced by considering two parts.
Firstly, we consider the part where the "positioned_vertices" dictionary is populated with vertices and their indexes. This yields a runtime of $O(N \times L)$ where N is the total number of layers and L is the number of vertices on each layer.
The second part calls the "up_sweep" and "down_sweep" algorithms. Considering the "down_sweep" algorithm, it iterates over the vertices on a layer which takes $O(L)$. The algorithm retrieves the incoming edges of each vertex and gets the indexes of the head vertices. This takes $O(|V| + |E|$ in the size of the input graph because each vertex and edge is visited exactly once. Calling the median takes $O(1)$ because it simply selects the middle element in the list if it is odd, otherwise it sums the two elements and divides them by 2. These operations take constant time.
Adjusting vertex positions takes $O(L)$ because the algorithm iterates through a layer and compares each vertex to the current vertex to see which ones are positioned to left and right. It also retrieves the maximum and minimum position of the left and right vertices respectively. This also takes $O(L)$ in the worst case where for example no vertex is positioned to the right of the current or the left. Therefore the runtime for adjusting vertex position is linear.
Overall, the coordinate assignment takes $O(|L(N + L(V + E))|)$.

7.2 Testing

Various tests were carried out to ensure the proper functioning of the application. Below discusses the tests that were performed.

Unit Tests

Unit testing occurred throughout the implementation process, to make sure functions functioned as intended. The assertions made in the tests focused on verifying the expected behaviour of the application's functions rather than implementation details. This approach also included post-implementation testing to ensure comprehensive coverage of various scenarios.

To ensure reliable unit testing, it was crucial to minimise dependencies between tests and

properly construct setup and tear down procedures. In addition, the tests were made to adhere to the following criteria:

- **Effectiveness**: Do the tests uncover potential errors or unexpected behaviour in the code? For example, a test was created to check the "read" method within the FileHandler class. JSON input that does not conform to the specification was used to verify whether the function correctly throws the expected error.
- **Efficiency**: Are the tests focused on isolating and testing a specific unit of code? For instance, in testing the Vertex class, a specific test was designed to evaluate the equality of two vertices. This essentially assesses the "__eq__" method within the vertex class.
- **Maintainability**: Can the tests be easily understood, modified, and debugged by other programmers? Clear class and method names were used throughout the tests, with additional comments provided where necessary. For instance, when testing the Vertex class, the test class was labeled as "TestVertex" containing methods such as "test_vertex_equality", which clearly indicate their purpose and function.
- **Coverage**: Do the tests cover a wide range of scenarios, including edge cases? For example, the test case "test_empty_json" examines a specific scenario where the input JSON file is empty.

At each milestone, the emphasis was on a working application. Equally important was the addition of essential tests and making sure that all tests passed without any issues. For example, when the graph module was added to represent graph data in JSON format, unit tests were created for its functions. Additionally, existing tests were updated to reflect the application's current state. These unit tests serve as a valuable safety net. If a test fails after a code change, particularly when adding a new component, it helps to identify the area requiring attention.

Manual Tests

These tests focused on the graphical user interface (GUI) and the visual representation of graphs. Key aspects evaluated were the readability of generated graphs, the application's ability to generate graphs of varying complexity, and overall user experience.

Testing procedures included running the application and conducting load tests. One load testing scenario involved generating TikZ code for a very large graph to assess the application's performance and ability to display complex graphs clearly.

Usability testing compared features to the defined requirements and evaluated their functionality under unusual input scenarios.

- RF1 & RF2: Users can add files to the application in two ways: clicking the "Add" button or selecting "File" > "Open" from the menu bar. The application prevents duplicate files from being added. If a file with the same path already exists in the list, the application adds the new file and removes the duplicate. When a large number of files are added, a scroll bar appears automatically to accommodate the file list within the interface window. This was tested by adding large number of files including existing ones.

- RF3, RF4 & RF5: These tests focused on the functionality of the "Generate Tikz Code" and the "Preview PDF" buttons for each added file. This included verifying the application's ability to generate TikZ code and displaying the corresponding PDF preview. Also, the tests explored how the application handles situations where a vertex label contains a lot of text, assessing potential impacts on the generated code and readability of the preview.

- RNF1: This requirement was tested by repeatedly generating TikZ code for the same JSON file. The test assessed whether the resulting graph remained consistent across multiple runs, meaning the vertex placement, edge layout, and overall structure of the graph remained identical.

- RNF2: This test evaluated the text area's ability to handle large graphs by loading a complex graph and verifying if the text area could display the code comfortably.

- RNF4: The application's functionalities were tested on Windows, macOS, and Linux operating systems to verify consistent behaviour across different platforms.

- RNF3 & RNF5: These tests assessed the application's functionality related to editing and saving generated code. Tests included:
  Editing the generated code and verifying if the changes were reflected in the preview.
  Saving the generated code in various file formats (e.g., .tex, .txt, .md) to ensure successful saving across different formats.

7.3 Limitations

The application's limitations can be grouped into two categories: generic limitations and limitations specific to the operating system.

Generic Limitations

- The readability of the graphs significantly decreases when long texts are used as vertex labels. Although the width of the vertices dynamically adjusts to accommodate the text size, this often results in vertex overlapping.

  When dealing with very large graphs (such as one tested with 80 vertices), the application's processing time increases significantly (approximately 10 seconds). In addition to the increase in processing times for such graphs, they are also prone to having overlapping vertices and an increased number of edge crossings, all of which significantly impact the readability of the graph.

- The current x-coordinate adjustment algorithm struggles to effectively place vertices as the graph widens, leading to overlapping vertices and uneven spacing.
  Flowcharts typically have a greater height than width. When the graph has a horizontal orientation, the algorithm rotates the original coordinates of vertices, resulting in a graph height becoming the width. This exacerbates the issue mentioned above, contributing to the weakness in vertex positioning.

- The application lacks the functionality to directly add labels to edges. While users can work around this by placing a vertex between two connected vertices and labeling that vertex, this approach increases the overall graph size and can potentially hinder readability.

Operating System-Specific Limitations

- **Windows**

  Tooltip functionality for buttons appears to be malfunctioning on the Windows operating system.

  The application relies on a bash script to compile the LaTeX file. This means that users on Windows will need to have Bash installed for successful compilation. [refer to appendix **??**]

- **MacOS**

  Currently, the application automatically opens the generated "tester.pdf" file after compilation on Windows and Linux operating systems. However, on macOS, users need to manually navigate to the root folder and open the file themselves.

7.4 Overall Project Evaluation

Referring to research papers and other background resources on this project was crucial. These resources helped clarify the project's objective, scope, and requirements, ultimately guiding the design process. While invaluable, some resources presented implementation challenges, particularly the vertex positioning algorithm. To ensure a thorough grasp of these algorithms, additional research was often necessary, to help break them down into simpler problems. This additional research sometimes led to longer-than-expected implementation times.

With regards to design, having a clearly defined class diagram and a graphical user interface(GUI) template provided a valuable model for the application. This eliminated the need to speculate about details such as the appearance of the application's GUI or the organisation of files.

In addition, planning the project with a minimum viable product (MVP) approach proved beneficial and allowed for flexibility. While further breaking down the project into smaller tasks with deadlines could have been ideal, this approach allowed me to focus on the challenges of complex implementations.

Despite planning for an early submission throughout the term, a combination of the lack of detailed task breakdowns and the time needed to fix functional and logical errors made the initial goal seem unachievable.

Looking towards the future, incorporating smaller task breakdowns with deadlines within the MVP approach could enhance project management and potentially allow for earlier submissions.

As outlined in the section 7.2, various testing methods were employed to verify the functionality of individual units within the application. Emphasis was placed on ensuring that, in the event of an error, the application gracefully exits. These tests also helped identify errors, particularly during the introduction of new features. While it is impractical to test every possible scenario, it is acknowledged that the application may not be entirely error-free. However, rigorous testing serves to minimise errors and aids detection should they occur.

Through dedicated effort, the project has delivered a functional application that successfully fulfills its main goal of simplifying TikZ graph creation.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

Since I had no prior experience with TikZ, a key aspect of this project involved learning the language to effectively translate input data into corresponding TikZ code. The learning process revealed that some TEX engines offer better stability than others. For instance, pdflatex proved more reliable than lualatex, meaning a file that compiles successfully with pdflatex might not compile with lualatex.This is because pdflatex has been around longer and is more thoroughly tested with a wider range of packages and document styles.

Given the project's focus on flowcharts, hierarchical graph layouts were crucial. Within this area, the Sugiyama framework emerged as a viable solution. This framework effectively combines several steps – cycle removal, layer assignment, vertex ordering, edge-crossing reduction, and vertex positioning – to generate readable flowcharts.

Delving deeper into each of these steps presented opportunities for problem-solving and experimentation, particularly regarding optimisation strategies. For example, in cycle removal, Gansner et al. [6, p.217] argued that reversing a small number of edges seems like a reasonable approach to potential optimisation. However, identifying this minimal set of edges (known as the "feedback arc set" problem) is computationally challenging and falls under the category of NP-complete problems. This highlights a potential area for further exploration.

During implementation, as detailed in section 5.3, it became obvious that using the graph drawing library hindered control over crucial aspects like vertex placement. While risk management plays a vital role, sometimes exploring entirely different approaches proves beneficial. In this case, considering alternatives led to a greater degree of control over graph, such as

positioning.

8.2 Future Work

Despite its functionality, there are opportunities to enhance the application by addressing the limitations discussed earlier and incorporating additional features. Here are some potential areas for improvement:

- The current vertex positioning utilises the DAG (Directed Acyclic Graph) [15, p.60] method to adjust x-coordinates. While implemented, this method can sometimes produce undesirable outcomes, such as vertex overlap or uneven spacing. To address these limitations, a heuristic approach introduced by Gansner et al. [6, p.222] offers a promising alternative.

  This heuristic approach starts with a good initial placement and iteratively improves it. It begins by assigning a coordinate of 0 to the leftmost nodes and packing them as far left as possible. Subsequent iterations involve applying a set of heuristics to refine the coordinate assignments. If an improvement is identified, the new coordinates are saved. The approach prioritises vertices based on their connections using a median heuristic.

- Another area for improvement concerns reducing the number of edge crossings, which can significantly impact the application's runtime efficiency. The current implementation relies on a brute-force approach for counting edge crossings. This approach can be computationally expensive, especially for large graphs. A more efficient approach could involve topological sorting. By performing a topological sort and analysing the order of vertices, potential edge crossings can be inferred, thus optimising the cross-reduction process.

- The project prioritised maintainability to ensure that adding new features would remain manageable. Incorporating edge labels and the concept of subgraphs into the graph are crucial future additions. However, these features introduce new challenges, such as label positioning to avoid overlap with vertices and edges, and ensuring subgraphs are small enough to fit within the overall graph layout.

Hierarchical graphs are prevalent across various sectors, such as computer science for modeling software structures. They offer a high-level overview of processes, making them valuable tools. Continued exploration of advanced layout and optimisation algorithms, along with their integration across different domains, holds the potential to further enhance the capabilities of hierarchical graphs.

References

[1] Directed acyclic graph. Directed acyclic graph. Online, 2023. accessed 20.11.23, URL https://en.wikipedia.org/wiki/Directed_acyclic_graph.

[2] Júlio César Batista. Diagram layout engines: Minimising hierarchical edge crossings. Online Blog, 2023. accessed 02.03.2024, URL https://www.terrastruct.com/blog/post/diagram-layout-engines-crossing-minimization/.

[3] Michelle A. Borkin, Azalea A Vo, Zoya Bylinskii, Phillip Isola, Shashank Sunkavalli, Aude Oliva, and Hanspeter Pfister. What makes a visualization memorable?, 2013. accessed 24.11.23, URL https://vcg.seas.harvard.edu/publications/what-makes-a-visualization-memorable/paper/.

[4] Data Interchange Format DIF. Data interchange format. online, updated 2023. accessed 24.11.23, URL https://en.wikipedia.org/wiki/Data_Interchange_Format.

[5] Python Documentation. Sorting techniques. Online. accessed: 02.04.2024, URL https://docs.python.org/3/howto/sorting.html.

[6] Emden R. Gansner, Stephen C. North, Eleftherios Koutsofios, and Kiem-Phong Vo. A technique for drawing directed graphs, 1993. accessed 24.11.23, URL https://www.researchgate.net/publication/3187542_A_Technique_for_Drawing_Directed_Graphs.

[7] Introducing JSON. Introducing json,ecma-404 the json data interchange standard. online, n.d. accessed 24.11.23, URL https://www.json.org/json-en.html.

[8] Olivia Kimber, Jennifer G. Cromley, and Katherine L. Molnar-Kimber. Let your ideas flow: Using flowcharts to convey methods and implications of the results in laboratory exercises, articles, posters, and slide presentations, 1900. accessed 24.11.23, URL https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5969414/.

[9] PGF/TikZ. Pgf/tikz. Online, 2023. accessed 14.11.23, URL https://en.wikipedia.org/wiki/PGF/TikZ.

[10] Jannis Pohlmann. Configurable graph drawing algorithms for the tikz graphics description language. Online pdf, 2011. accessed 02.11.23, URL http://www.tcs.uni-luebeck.de/downloads/papers/2011/2011-10-14-configurable-graph-drawing-algorithms-jannis-pohlmann.pdf.

[11] Martin Scharrer. The standalone class and package. online pdf, 2010. accessed 23.11.23, URL https://www.latex-project.org/about/,https://www.sys.kth.se/docs/texlive/texmf-dist/doc/latex/standalone/standalone.pdf.

[12] Topological Sorting. Topological sorting. Online. accessed: 02.12.2023, URL https://en.wikipedia.org/wiki/Topological_sorting.

[13] Roberto Tamassia. Handbook of graph drawing and visualization. CRC Press, 2013. accessed 22.11.23, URL https://cs.brown.edu/people/rtamassi/gdhandbook/chapters/hierarchical.pdf.

[14] Till Tantau and Jannis Pohlmann. Graph drawing algorithms: Layered layouts. Online pdf, 2023. accessed 18.11.23, URL https://tikz.dev/gd-layered.

[15] Thomas Wuerthinger, Christian Wimmer, and Hanspeter Mössenböck. Visualization of program dependence graphs. volume 4959, pages 60–61, 03 2008. accessed: 04.03.2024, URL https://www.researchgate.net/publication/221302634_Visualization_of_Program_Dependence_Graphs.